

Name of the Student	SHAM S.
Reg. No	192325076
GitHub Link	https://github.com/Sham-2005/MLA0403-Deep_learning

SIMATS ENGINEERING

CO2 - ASSESSMENT TOOL 2

Scenario-Based Questions

COURSE NAME: Deep Learning

COURSE CODE: MLA04

COURSE OUTCOME COVERED

CO 2:Students will be able to apply supervised learning algorithms such as linear regression, classification models, decision trees, neural networks, support vector machines, and ensemble methods to solve real-world prediction and classification problems. They will gain the ability to select appropriate models, train them using datasets, and evaluate their performance. Students will also understand the strengths and limitations of different supervised learning approaches.(BTL-4)

CO Weightage : 15%

Assessment Tool Weightage: 40%

Duration: 90 minutes

Marks: 25

Code of Conduct:

I SHAM S. (192325076) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Name of the student:

Scenario-Based Questions

CO-AT2

QUESTIONS: 05

Total Marks:25

1. Unsupervised Training of Neural Networks, Auto Encoders

A hospital wants to identify abnormal ECG patterns from thousands of unlabeled records. Which deep learning technique from the syllabus would you recommend? Justify your choice and explain the workflow.

Recommended Technique: Autoencoder

For identifying **abnormal ECG patterns from thousands of unlabeled records**, I would recommend an **Autoencoder**, which is an unsupervised deep learning technique.

Why Autoencoder?

- ECG records are **unlabeled**, so supervised methods cannot be directly used without manually labeling normal and abnormal ECGs.
- An autoencoder can learn the **normal pattern of ECG signals** from the available data.
- When an abnormal ECG is given, the trained model will usually reconstruct it poorly.
- The difference between the **original ECG and reconstructed ECG** is called the **reconstruction error**.
- A high reconstruction error can indicate an **abnormal ECG pattern**.

Workflow

1. Collect ECG Data

Thousands of unlabeled ECG records are collected.

2. Preprocessing

Remove noise, normalize the ECG signals, and divide them into suitable time windows.

3. Train the Autoencoder

The ECG signal is given as input to the autoencoder.

4. Encoder

The encoder compresses the ECG into a smaller representation called the **latent representation**.

ECG Input → Encoder → Latent Representation

5. Decoder

The decoder reconstructs the ECG from this compressed representation.

Latent Representation → Decoder → Reconstructed ECG

6. Calculate Reconstruction Error

Compare the original ECG with the reconstructed ECG using an error measure such as **Mean Squared Error (MSE)**.

where:

- = original ECG value

- = reconstructed ECG value

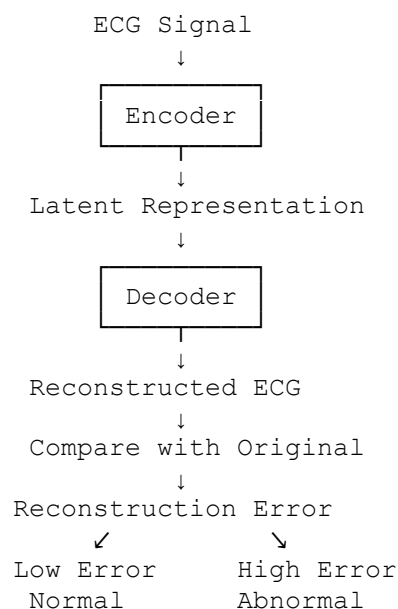
7. Set an Anomaly Threshold

A threshold is selected based on reconstruction errors from normal ECG patterns.

8. Detect Abnormal ECGs

- **Low reconstruction error** → **Normal pattern**
- **High reconstruction error** → **Potential abnormality**

Simple Architecture



Conclusion

An **Autoencoder** is suitable because it can learn useful representations from **unlabeled ECG data**. By learning to reconstruct normal ECG patterns and measuring reconstruction error, the hospital can automatically identify **potentially abnormal ECG records** without requiring thousands of manually labeled examples

2. Auto Encoders, Representation Learning

A retail company wants to reduce the dimensionality of customer purchase data before performing customer segmentation. Explain how an autoencoder can be used to solve this problem.

2. Autoencoders – Representation Learning

An **Autoencoder** can be used to reduce the dimensionality of customer purchase data by learning a compact representation of the important features while removing unnecessary information.

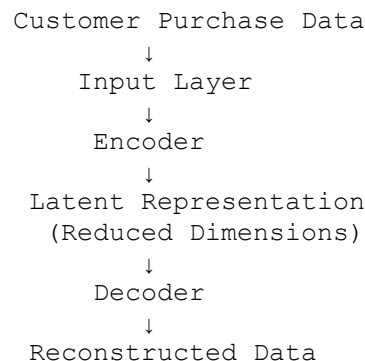
How it works

Suppose the retail company has customer data such as:

- Purchase frequency
- Total spending
- Product categories purchased
- Average order value
- Discount usage
- Number of orders
- Online/offline purchases

This data may contain many features, making customer segmentation more complex.

Autoencoder Architecture



Workflow

1. Input Data

The customer's purchase data is given to the autoencoder as input.

For example:

2. Encoder

The encoder gradually reduces the number of dimensions and learns the most important patterns in the data.

For example:

100 Features → 50 Features → 20 Features → 5 Features

The 5-dimensional representation is called the **latent representation** or **encoding**.

3. Representation Learning

The autoencoder learns which features and combinations of features are important for representing customer behavior.

For example, the latent features might implicitly represent:

- High-value customers
- Frequent buyers
- Discount-sensitive customers
- Category preferences
- Occasional buyers

These are learned automatically rather than manually defining them.

4. Decoder

The decoder takes the compressed representation and attempts to reconstruct the original customer data.

5-Dimensional Representation → Decoder → Original 100 Features

The model is trained by minimizing the difference between the original and reconstructed data.

where x is the original data and $\hat{x}$ is the reconstructed data.

5. Extract the Reduced Data

After training, the **decoder is no longer required** for segmentation.

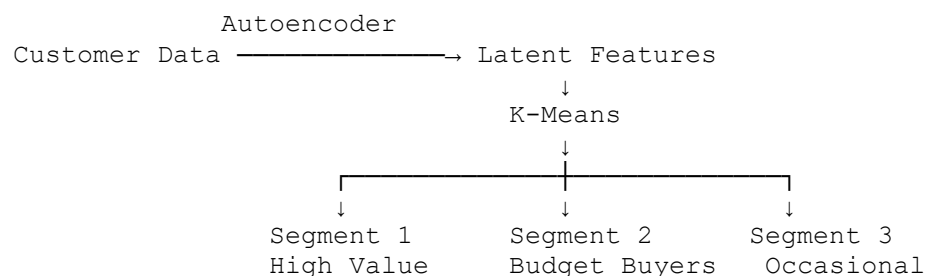
The company uses only the encoder:

```
Customer Data
  ↓
Encoder
  ↓
5-Dimensional Representation
  ↓
Customer Segmentation
```

6. Perform Clustering

The reduced representations can then be given to a clustering algorithm such as **K-Means**.

For example:



Advantages

- Reduces a large number of features into a smaller representation.
- Removes redundant and noisy information.
- Learns useful features automatically.
- Makes clustering faster and easier.
- Can improve the quality of customer segmentation.
- Handles nonlinear relationships better than traditional linear dimensionality reduction methods such as PCA.

Conclusion

An **autoencoder** performs **representation learning** by **compressing high-dimensional customer purchase data into a smaller latent representation while retaining important information**. The reduced representation can then be used with clustering algorithms such as K-Means to create meaningful customer segments.

3. Width and Depth of Neural Networks, Activation Functions

A facial recognition system performs poorly when trained using a shallow neural network. Recommend architectural changes involving network depth and activation functions.

3. Width and Depth of Neural Networks – Activation Functions

A **shallow neural network** has only a small number of hidden layers, so it may not be able to learn the complex patterns required for facial recognition. The system can be improved by increasing the **depth and width** of the network and using appropriate **activation functions**.

Recommended Architectural Changes

1. Increase Network Depth

Instead of using only one or two hidden layers, use **multiple hidden layers**.

```

Input Image
  ↓
Hidden Layer 1
  ↓
Hidden Layer 2
  ↓
Hidden Layer 3
  ↓
Hidden Layer 4
  ↓
Output Layer

```

Deeper networks can learn hierarchical features:

- Early layers → edges and basic shapes

- Middle layers → eyes, nose, mouth, and other facial features
- Deeper layers → complex facial structures and identity-related patterns

Thus, increasing **depth improves the network's ability to learn complex representations.**

2. Increase Network Width

Increase the number of neurons/filters in appropriate layers.

For example:

```
Layer 1 → 64 neurons
Layer 2 → 128 neurons
Layer 3 → 256 neurons
Layer 4 → 512 neurons
```

A wider network can learn more features simultaneously. However, excessive width can increase computational cost and may cause overfitting.

3. Use ReLU Activation

Use **ReLU (Rectified Linear Unit)** in hidden layers:

```
[
ReLU(x)=max(0,x)
]
```

Advantages:

- Simple and computationally efficient
- Helps reduce the vanishing-gradient problem
- Allows deep networks to train more effectively
- Introduces non-linearity

For example:

```
Input
↓
Dense/Conv Layer
↓
ReLU
↓
Dense/Conv Layer
↓
ReLU
↓
...
```

4. Consider Leaky ReLU

If neurons become inactive because they always produce zero for negative inputs, **Leaky ReLU** can be used.

```
[
f(x)=
\begin{cases}
x & x>0\\
\alpha x & x\leq 0
\end{cases}
]
```

It allows a small gradient for negative values and helps avoid the **dying ReLU problem**.

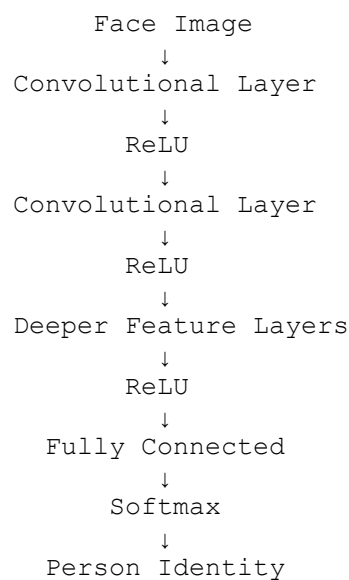
5. Use an Appropriate Output Activation

For facial identification among multiple known people, use **Softmax** in the output layer.

```
[
Softmax(z_i)=\frac{e^{z_i}}{\sum_j e^{z_j}}
]
```

It produces a probability for each person/class.

Recommended Architecture



Conclusion

The poor performance of the shallow network can be addressed by **increasing depth to learn hierarchical facial features, increasing width to learn more feature patterns, and using ReLU/Leaky ReLU activation functions to provide effective non-linearity and gradient flow**. For classification, Softmax can be used at the output layer.

4. Restricted Boltzmann Machines (RBMs), Unsupervised Training of Neural Networks

A streaming service wants to recommend movies to users based on their viewing history without using labeled data. Explain how RBMs can be applied to build the recommendation engine.

4. Restricted Boltzmann Machines (RBMs) – Recommendation System

A **Restricted Boltzmann Machine (RBM)** is an unsupervised neural network that can learn hidden patterns from user–movie interaction data. It is suitable for a streaming service because movie ratings or viewing history can be used without requiring manually labeled training data.

1. Represent the Viewing History

Create a **user–movie interaction matrix**.

For example:

User	Movie A	Movie B	Movie C	Movie D
User 1	1	1	0	0
User 2	0	1	1	0
User 3	1	0	0	1

Here:

- **1** → User watched/liked the movie
- **0** → User did not watch/interact with the movie

This matrix becomes the input to the RBM.

2. RBM Architecture

An RBM contains two layers:

Visible Layer Hidden Layer

Movie A ● ————— ● Hidden 1

Movie B ● ————— ● Hidden 2

Movie C ● ————— ● Hidden 3

Movie D ● ————— ● Hidden 4

No connections within the same layer

- **Visible layer:** represents movies and the user's viewing history.
- **Hidden layer:** learns hidden preferences or patterns.
- There are connections between visible and hidden layers, but **no connections within a layer**.

3. Unsupervised Training

The RBM learns patterns from the viewing data without requiring labels.

For example, it may discover that users who watch:

Action + Sci-Fi → often also watch Superhero movies

or:

Romance + Drama → often watch Romantic Comedy

These relationships are learned by the hidden units.

4. Learning Process

The RBM is commonly trained using **Contrastive Divergence (CD)**.

The basic process is:

User Viewing History



Visible Layer



Hidden Layer



Reconstruct Movie Preferences



Compare with Original Input



Update Weights



Repeat

During training, the model adjusts its weights so that it can reconstruct the user's observed movie preferences accurately.

5. Generate Recommendations

After training, suppose User 1 has watched:

Movie A = 1

Movie B = 1

Movie C = 0

Movie D = 0

The RBM uses the user's viewing history to activate hidden preference patterns and calculates probabilities for unseen movies.

For example:

Movie C $\rightarrow$ 0.82

Movie D $\rightarrow$ 0.35

Movie E $\rightarrow$ 0.76

Movie F $\rightarrow$ 0.20

The system can recommend movies with the highest predicted probabilities:

Movie C $\rightarrow$ Movie E $\rightarrow$ Movie D

Overall Workflow

User Viewing History



User–Movie Matrix



Visible Layer



Hidden Layer



Learn User Preferences



Predict Unseen Movies



Rank Movies



Personalized Recommendations

Advantages of RBMs

- Works with **unlabeled user behavior data**.
- Learns hidden patterns in viewing preferences.
- Can handle large user–movie interaction datasets.
- Provides personalized recommendations.
- Can discover relationships between different movies and user preferences.

Conclusion

An **RBM can learn hidden user preferences from viewing history without labeled data**. The visible layer represents movies watched by the user, while the hidden layer learns latent preference patterns. After training, the RBM predicts the user's interest in unseen movies and recommends those with the highest predicted preference scores.

4. Restricted Boltzmann Machines (RBMs) – Recommendation System

A **Restricted Boltzmann Machine (RBM)** is an unsupervised neural network that can learn hidden patterns from user–movie interaction data. It is suitable for a streaming service because movie ratings or viewing history can be used without requiring manually labeled training data.

1. Represent the Viewing History

Create a **user–movie interaction matrix**.

For example:

User	Movie A	Movie B	Movie C	Movie D
User 1	1	1	0	0

User	Movie A	Movie B	Movie C	Movie D
User 2	0	1	1	0
User 3	1	0	0	1

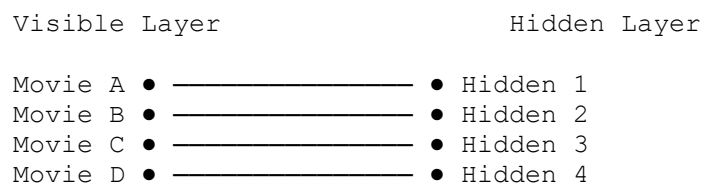
Here:

- **1** → User watched/liked the movie
- **0** → User did not watch/interact with the movie

This matrix becomes the input to the RBM.

2. RBM Architecture

An RBM contains two layers:



No connections within the same layer

- **Visible layer:** represents movies and the user's viewing history.
- **Hidden layer:** learns hidden preferences or patterns.
- There are connections between visible and hidden layers, but **no connections within a layer**.

3. Unsupervised Training

The RBM learns patterns from the viewing data without requiring labels.

For example, it may discover that users who watch:

Action + Sci-Fi → often also watch Superhero movies

or:

Romance + Drama → often watch Romantic Comedy

These relationships are learned by the hidden units.

4. Learning Process

The RBM is commonly trained using **Contrastive Divergence (CD)**.

The basic process is:

User Viewing History

```

      ↓
Visible Layer
      ↓
Hidden Layer
      ↓
Reconstruct Movie Preferences
      ↓
Compare with Original Input
      ↓
Update Weights
      ↓
Repeat

```

During training, the model adjusts its weights so that it can reconstruct the user's observed movie preferences accurately.

5. Generate Recommendations

After training, suppose User 1 has watched:

```

Movie A = 1
Movie B = 1
Movie C = 0
Movie D = 0

```

The RBM uses the user's viewing history to activate hidden preference patterns and calculates probabilities for unseen movies.

For example:

```

Movie C → 0.82
Movie D → 0.35
Movie E → 0.76
Movie F → 0.20

```

The system can recommend movies with the highest predicted probabilities:

Movie C → Movie E → Movie D

Overall Workflow

```

User Viewing History
      ↓
User-Movie Matrix
      ↓
  Visible Layer
      ↓
  Hidden Layer
      ↓
Learn User Preferences
      ↓
Predict Unseen Movies
      ↓
Rank Movies
      ↓
Personalized Recommendations

```

Advantages of RBMs

- Works with **unlabeled user behavior data**.
- Learns hidden patterns in viewing preferences.
- Can handle large user–movie interaction datasets.
- Provides personalized recommendations.
- Can discover relationships between different movies and user preferences.

Conclusion

An **RBM can learn hidden user preferences from viewing history without labeled data**. The visible layer represents movies watched by the user, while the hidden layer learns latent preference patterns. After training, the RBM predicts the user's interest in unseen movies and recommends those with the highest predicted preference scores.

5. Deep Learning Applications, Representation Learning

A manufacturing company wants to detect defective products from sensor readings collected every second. Design a deep learning solution using concepts from representation learning and neural networks.

5. Deep Learning Applications – Representation Learning

A manufacturing company can use a **deep neural network with representation learning** to automatically learn patterns from sensor readings and identify defective products.

1. Collect Sensor Data

Sensors collect readings every second, such as:

- Temperature
- Pressure
- Vibration
- Speed
- Voltage
- Humidity
- Motor current

Each product will have a sequence of sensor readings.

2. Preprocess the Data

Before training:

- Remove or handle missing values.
- Remove extreme noise.
- Normalize sensor values.
- Divide continuous readings into fixed time windows.
- Assign labels such as **Normal** or **Defective** if historical labeled data is available.

Example:

Time →	1s	2s	3s	4s	5s
Temp →	70	72	71	85	90
Pressure →	10	11	10	15	17
Vibration→	2.1	2.2	2.0	5.8	6.2

3. Representation Learning

Instead of manually deciding which sensor patterns indicate defects, the neural network **automatically learns useful representations** from the raw sensor data.

For example:

```

Raw Sensor Data
      ↓
Feature Extraction Layers
      ↓
Learned Representation
      ↓
Defect Classification
  
```

The network may learn combinations such as:

High temperature + increasing vibration + abnormal pressure → possible defect

4. Use a Deep Neural Network

A suitable architecture for time-series sensor data is a **1D CNN**, or a combination of **CNN + LSTM**.

Example:

```

Sensor Time-Series Data
      ↓
    1D CNN
      ↓
Feature Extraction
      ↓
    LSTM
      ↓
Temporal Representation
      ↓
Fully Connected
      ↓
    Softmax
    ↙    ↘
Normal Defective
  
```

5. Role of the CNN

The **1D CNN** detects local patterns in sensor readings.

For example, it can identify:

- Sudden temperature increases
- Vibration spikes
- Pressure fluctuations
- Repeated abnormal patterns

6. Role of LSTM

An **LSTM (Long Short-Term Memory)** network can learn dependencies across time.

For example, it can understand that:

```
Increasing temperature
      ↓
Increasing vibration
      ↓
Pressure fluctuation
      ↓
Product becomes defective
```

Thus, the model considers not only individual sensor values but also **how they change over time**.

7. Classification

The final layer classifies the product:

```
[
Output =
\begin{cases}
Normal\
Defective
\end{cases}
]
```

For multiple types of defects, the output layer can contain multiple classes, such as:

```
Normal
Overheating
Excessive Vibration
Pressure Failure
Motor Fault
```

8. Training

The model is trained using historical sensor data.

The training process is:

```
Sensor Data + Labels
    ↓
Deep Neural Network
    ↓
Prediction
    ↓
Calculate Loss
    ↓
Backpropagation
    ↓
Update Weights
    ↓
Repeat
```

9. Real-Time Detection

Once trained, the model can process new sensor readings every second.

```
New Sensor Reading
    ↓
Preprocessing
    ↓
Trained Model
    ↓
Defect Probability
    ↓
Normal / Defective
```

If the probability of a defect exceeds a predefined threshold, the system can generate an **early warning**.

Conclusion

A **CNN-LSTM deep learning architecture** is an effective solution for detecting defective products from continuously collected sensor readings. The CNN automatically learns important local sensor patterns, while the LSTM learns their temporal relationships. This demonstrates **representation learning**, because the system automatically learns useful features from raw sensor data instead of relying entirely on manually engineered features.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
----------	-----------	---------------------	----------------------	----------------------	---------------------

Concept Identification	25%	Identifies all key concepts from literature accurately and comprehensively	Identifies most key concepts with minor omissions	Identifies limited concepts; some are irrelevant or missing	Fails to identify essential concepts
Linkages	25%	Establishes clear, meaningful, and	Shows correct relationships	Relationships are weak,	No meaningful

		accurate relationships between concepts	with minor gaps	unclear, or partially incorrect	relationships established
Organization	20%	Concepts are wellstructured hierarchically with logical flow and grouping	Mostly organized with minor structural issues	Some organization but lacks clear hierarchy	No logical organization or structure
Integration of Literature	20%	Integrates multiple studies effectively to show connections and comparisons	Shows some integration of literature	Limited integration; mostly isolated concepts	No integration of literature evidence
Clarity	10%	Concept map is clear, neat, visually structured, and easy to interpret	Generally clear with minor readability issues	Clarity is reduced; difficult to interpret in parts	Poorly presented and hard to understand